当代人工智能实验二：搜索算法的实现、性能分析与优化探索

仓库链接：https://github.com/winkwinkk/contemporary-artificial-intelligence-experiments

一、实验目标与算法设计

1.1 实验目标

本实验围绕搜索算法的实现与性能优化展开，完成普通迷宫寻路、加权地形寻路、博弈树搜索和综合宝藏博弈四项任务。通过实现 A*、Minimax 和 Alpha-Beta 算法，进一步比较不同启发函数、搜索深度及节点访问顺序对搜索效率和决策结果的影响，并通过自设场景验证优化策略的适用条件。

1.2 算法设计与实现

实验主要实现以下算法：

算法	核心原理与实现
A* 搜索	将地图坐标作为搜索状态，使用 f(n)=g(n)+h(n) 评价节点，通过最小优先队列选择待扩展节点，并利用父节点记录恢复路径。
加权 A*	将进入目标格子的实际代价累加至 g(n)，结合启发函数搜索最小总代价路径，并通过代价更新维护当前最优路径。
Minimax	通过递归搜索博弈树，MAX 节点选择最大值，MIN 节点选择最小值，并将叶节点评价值逐层回传。
Alpha-Beta	在 Minimax 基础上维护 α 和 β 界限，当 $\alpha \geq \beta$ 时剪去不影响最终决策的剩余分支。

其中，题目三以给定博弈树的根节点为 MAX 节点、叶节点的 value 为评价值；题目四进一步将路径规划与竞争博弈结合，通过 A* 预先计算关键位置之间的最小移动代价，再利用 Minimax 和 Alpha-Beta 进行决策。

1.3 实验设计与评价指标

实验采用控制变量的对照方式，在相同地图、博弈树或初始状态下改变启发函数、搜索深度或节点访问顺序，比较不同策略的性能表现。具体包括：题一比较零启发与 Manhattan 启发在不同障碍密度下的搜索效果；题二验证修正启发函数的性质，并比较不同平局策略及多地图下的优化效果；题三比较不同子节点访问顺序对 Alpha-Beta 剪枝效率的影响；题四分析不同搜索深度和宝藏访问顺序对博弈搜索的影响。

主要评价指标包括：

- 搜索效率： 访问节点数、扩展节点数和生成节点数。
- 运行开销： 平均运行时间及中位数。
- 搜索质量： 最终路径代价、最优博弈值及最优行动。
- 剪枝效果： 剪枝分支数、剪枝发生次数及节点减少比例。

二、题目一：A* 最短路径搜索

2.1 启发函数对搜索效率的影响

在四方向移动、每步代价为 1 的地图中，采用 A* 算法寻找最短路径。为验证启发函数对搜索效率的影响，以零启发函数 h(n)=0 为基线，与 Manhattan 距离启发函数进行对照。本实验关注两个问题：Manhattan 启发能否在保持最短路径结果的同时减少搜索节点？地图障碍密度变化是否会影响启发函数的优化收益？

2.2 障碍密度拓展实验

为避免结论仅依赖单张地图，进一步设计 20×20 随机地图实验，设置 10%、20%、30%三种障碍生成概率，每种密度保留 10 张可达地图，两种方法在每张地图上分别重复运行 5 次。实验共包含 30 张地图、300 次正式搜索，并形成 60 条按地图和方法聚合的记录。

障碍率	h=0 节点	Manhattan 节点	h=0 时间(ms)	Manhattan 时间(ms)	路径长
10%	361.8	321.1	1.913	1.836	38.0
20%	318.7	226.1	1.559	1.187	38.0
30%	261.7	140.6	1.181	0.733	39.2

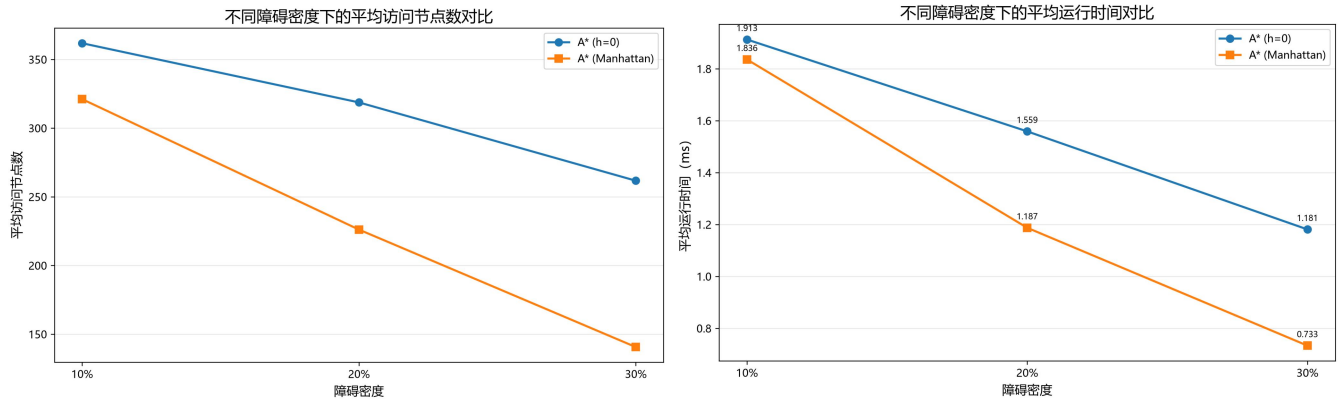


图 2-1 不同障碍密度下 A 的访问节点数与运行时间对比*

Manhattan 在三种障碍密度下均减少了访问节点，同时保持相同的路径长度。随着障碍生成概率增加，本次样本中 Manhattan 的节点削减比例由 11.25% 提高至 46.27%，说明启发函数在这些地图上能够有效缩小搜索范围。运行时间也由 h=0 的 1.913、1.559、1.181 ms 分别降至 1.836、1.187、0.733 ms，降幅约为 4.0%、23.9% 和 38.0%，与节点数量变化趋势一致。

但障碍概率增加时，两种算法的平均访问节点数反而下降。这是因为实验只统计成功生成的可达地图，实际搜索规模还受到地图连通结构、障碍位置和可行路径分布的影响，不能据此认为障碍越多，寻路越容易。

本实验表明，在单位移动代价和四方向移动条件下，Manhattan 启发能够在保持最短路径正确性的同时减少搜索节点，但其优化幅度与地图结构有关。

三、题目二：加权地形 A* 与启发函数优化

3.1 代价建模与启发函数修正

题目二将单位移动代价改为进入目标格子的代价，障碍格不可通行，终点 E 的进入代价为 0，路径总代价为各步进入代价之和。因此考虑到普通 Manhattan 距离能否直接用于该加权地图，以及如何设计满足最优性要求的启发函数的问题。

实验发现，普通 Manhattan 距离可能高估实际剩余代价。例如标准地图中，最优路径总代价为 7，而起点到终点的 Manhattan 距离为 8，因此不能直接保证其可采纳性。考虑到当前地图中进入终点 E 的代价为 0，其他相关可通行格子的进入代价至少为 1，设计修正启发函数：**h(n)=max(0,d_M(n,E)-1)**

以标准地图为例，算法返回的最优路径依次进入 8 个格子，其进入代价分别为 1、1、1、1、1、1、1、0，累计总代价为 7，与 Dijkstra 基线结果一致。

通过实际路径代价与启发值对照，以及相邻节点的一致性检查，验证修正启发函数在当前实验地图中的可采纳性和一致性。在此基础上，进一步比较三种启发方式的搜索结果，发现它们均得到相同的最小总代价，而修正后的启发函数减少了部分搜索节点，说明其在当前实验条件下能够兼顾搜索结果的正确性与搜索效率。

3.2 平局策略优化

在启发函数修正后，进一步研究当多个节点具有相同的 f(n) 值时，访问顺序是否会影响搜索效率。

设计两种平局处理策略：

- FIFO：按照进入优先队列的先后顺序处理。
- Lower-H：相同 f 值时优先选择启发值 h 较小的节点。

在三种固定地形下进行对照：

地形	FIFO 扩展节点	Lower-H 扩展节点	节点减少比例
均匀地形	99	18	81.82%
高代价地形	83	18	78.31%
随机地形	82	81	1.22%

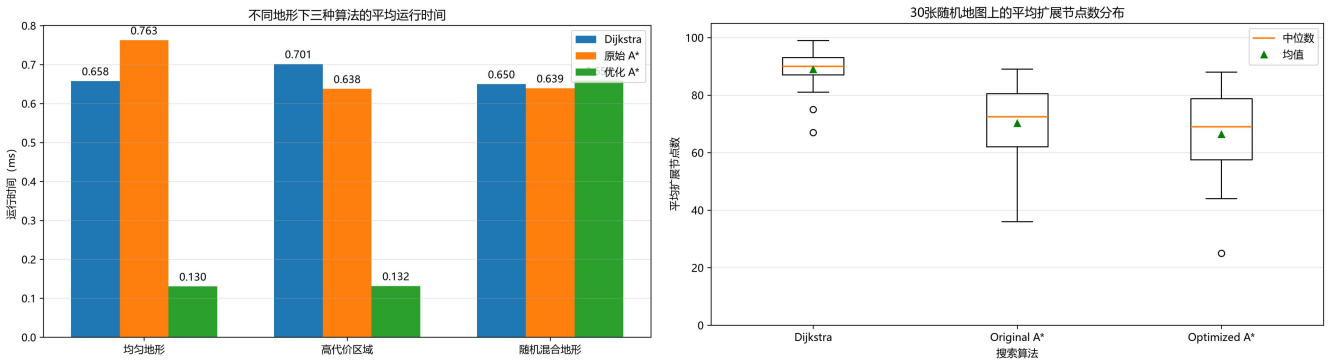


图 3-1 固定地形与多地图实验的搜索效率对比

Lower-H 在均匀和高代价地形中取得明显的节点削减效果，但在随机地形中收益很小。这说明即使启发函数保持不变，节点访问顺序仍会影响搜索过程，而且排序收益与地图代价分布有关。

3.3 多地图拓展验证

为检验固定地形实验中的优化效果是否具有一定稳定性，进一步设计 30 张 10×10 随机地形地图，采用 1、3、8 三种数值权值，在相同地图上比较 Dijkstra、原始 A* 和优化 A*（Lower-H），每种方法每张地图重复 5 次。

方法	平均扩展节点数
Dijkstra	88.93
原始 A*	70.23
优化 A*（Lower-H）	66.40

三种方法在相同地图上得到一致的最小路径代价。与原始 A* 相比，优化 A* 的平均扩展节点数进一步减少约 5.46%。箱线图同时显示，30 张地图上的搜索开销存在一定离散程度，说明优化收益并非在每张地图上完全一致。与固定地形实验中部分条件下的大幅节点削减相比，多地图实验中的平均收益较小，表明单一地图上的优化结果不能直接代表不同地图上的普遍表现，节点排序的实际收益仍受到地图结构和代价分布影响。

四、题目三：Minimax 与 Alpha-Beta 剪枝及搜索顺序优化

4.1 算法原理与实现

题目三采用 Minimax 算法计算博弈树的最优决策值。根节点为 MAX 节点，MAX 选择子节点中的最大值，MIN 选择子节点中的最小值，并将叶节点的评价值逐层向根节点回传。

Alpha-Beta 在 Minimax 基础上引入两个边界：α 表示 MAX 当前能够保证的下界，β 表示 MIN 当前能够保证的上界。当 α ≥ β 时，剩余分支不可能影响最终决策，因此可以提前停止搜索。

在给定的标准测试树中，三个 MAX 子树分别回传 5、9、2，两个 MIN 子树分别回传 5、2，根节点最终得到最优值 5。Minimax 与 Alpha-Beta 得到相同的最优博弈值，说明剪枝没有改变搜索结果，但能够减少不必要的节点访问。

4.2 节点排序实验与分析

在确认两种算法能够得到一致结果后，进一步研究子节点访问顺序对 Alpha-Beta 剪枝效率的影响。由于 Alpha-Beta 的剪枝效果与当前搜索过程中 α、β 边界的更新有关，因此即使博弈树和搜索深度相同，不同的访问顺序也可能产生不同的搜索开销。

为此，设计三种搜索策略进行对照：

- FIFO： 按照原始子节点顺序搜索，作为基础策略。
- Reverse： 反转子节点访问顺序，观察顺序变化对剪枝效果的影响。
- Oracle： 利用子树真实 Minimax 值进行理想排序，作为理论参照。

实验设置搜索深度为 3、4、5、6，分支因子为 2、3、4。每种条件生成 30 棵随机博弈树，并在相同博弈树上比较 Minimax、FIFO、Reverse 和 Oracle 四种方法，每种方法重复运行 5 次。实验共包含 360 棵随机博弈树，形成 7200 条详细运行记录。

以搜索深度 6、分支因子 4 为例，实验结果如下：

算法	平均访问节点	搜索时间(ms)	预计算时间(ms)	总时间(ms)
Minimax	5461.0	5.925	0	5.925
Alpha-Beta FIFO	1462.8	2.213	0	2.213
Alpha-Beta Reverse	1418.1	2.285	0	2.285

Alpha-Beta Oracle	268.0	0.618	9.684	10.302

从结果可以看出，Alpha-Beta 相比 Minimax 能够有效减少搜索节点。FIFO 将平均访问节点数从 5461.0 降至 1462.8，减少约 73.22%，说明即使不进行额外排序，剪枝机制也能显著降低搜索开销。Reverse 在该条件下进一步将节点数减少至 1418.1，但平均搜索时间由 2.213ms 增加至 2.285ms。这说明节点访问数量减少并不一定带来实际运行速度提升，搜索过程中的其他开销和运行环境波动也会影响计时结果。因此，不能仅依据节点数量判断一种排序策略是否更优。

同时可以发现，采用 Oracle 排序后，Alpha-Beta 能够进一步减少搜索节点。其正式搜索平均访问节点数仅为 268.0，相比 FIFO 减少约 81.68%。但是，Oracle 需要预先计算子树真实 Minimax 值，正式搜索之外还产生了 9.684 ms 的预计算开销。将这部分成本纳入后，总运行时间达到 10.302 ms，明显高于 FIFO。因此，Oracle 虽然能够显著减少正式搜索节点，但依赖额外的真实子树计算，不能直接视为实际可部署的加速方案。其价值主要在于提供理想排序的效果参照，帮助分析节点访问顺序对 Alpha-Beta 剪枝能力的影响。

综合来看，Alpha-Beta 的搜索效率不仅取决于剪枝机制本身，也受到子节点访问顺序的影响。Reverse 的收益并不稳定，而 Oracle 的节点削减效果又受到预计算成本的限制。这使我认识到，评价搜索优化不能只关注节点数量，还需要综合考虑排序开销与完整运行时间。后续可以进一步探索不依赖真实子树值、计算成本更低的启发式节点排序方法。

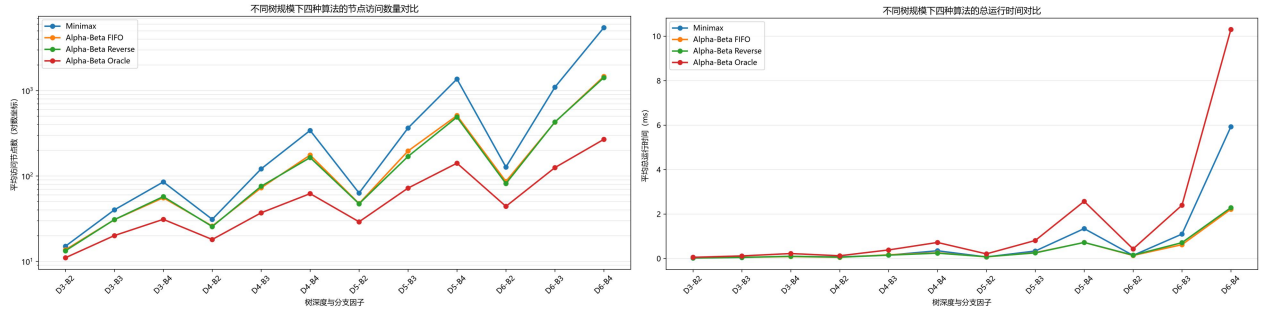


图 4-1 不同树规模下四种算法的节点访问量与总运行时间对比

图 4-1 进一步展示了不同树规模下的整体变化趋势。随着博弈树规模增大，各算法的搜索开销总体上升，不同排序策略之间的效率差异也逐渐显现。Oracle 虽然能够显著减少搜索节点，但预计算成本限制了其实际加速效果。

五、题目四：竞争环境下的宝藏分配

5.1 问题建模与状态设计

本题将地图寻路与双人博弈结合。玩家 A 为 MAX，玩家 B 为 MIN，双方按照 A→B→A→B 的顺序轮流选择四个宝藏。每次选择后，玩家移动至对应位置，并获得宝藏价值减去移动代价的收益。首先，使用加权 A* 计算六个关键位置（A、B、T1～T4）两两之间的最小移动代价，构建距离矩阵，如表 5-1 所示。

表 5-1 六个关键位置之间的最小移动代价矩阵

位置	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

在此基础上，将博弈状态表示为双方当前位置、剩余宝藏、累计收益和当前行动玩家组成的状态。初始状态为 (A, B, {T1,T2,T3,T4}, 0, 0, A)。每次行动后，根据距离矩阵计算移动代价，更新玩家位置与累计收益，并移除已获取的宝藏。单次行动收益及状态评价函数定义为：

$$Gain(p, T_i) = V_i - d(p, T_i)$$

$$U(s) = Score_A - Score_B$$

其中，V_i 为宝藏价值，$d(p,T_i)$ 为当前位置到目标宝藏的最小移动代价。A 选择使评价值最大的行动，B 选择使评价值最小的行动。通过 Minimax 与 Alpha-Beta 搜索，比较不同搜索深度下的最优决策及搜索开销。

5.2 搜索深度与决策结果

在 H=1、2、3、4 四种搜索深度下，对两种算法进行对照，每种条件预热 20 次、正式重复 1000 次。

深度	Minimax 值/行动	Alpha-Beta 值/行动	Minimax 节点	AB 节点

H=1	2 / T2	2 / T2	5	5
H=2	-1 / T2	-1 / T2	17	13
H=3	2 / T2	2 / T2	41	26
H=4	3 / T3	3 / T3	65	47

两种算法在所有搜索深度下均得到一致的博弈值和最优行动。H=2 与 H=4 分别得到题目要求的 T2、-1 和 T3、3 标准结果。随着搜索深度增加，搜索空间扩大，Alpha-Beta 通过剪枝减少访问节点。H=3 时节点减少约 36.59%，H=4 时减少约 27.69%。

5.3 节点排序优化与进一步探索

为进一步研究节点访问顺序对 Alpha-Beta 剪枝效率的影响，设计三种排序策略进行对照：FIFO 按原始顺序访问子节点；Reverse 按相反顺序访问；Greedy 根据当前玩家的收益情况优先选择预期更有利的宝藏，以改善搜索顺序。

实验设置搜索深度 H=2、3、4，每种策略预热 20 次并重复运行 1000 次，比较访问节点数、剪枝情况和运行时间。

以 H=4 为例，三种策略的实验结果如下：

排序策略	平均访问节点	平均运行时间(ms)
FIFO	47	0.172602
Reverse	46	0.187488
Greedy	37	0.195948

从结果可以看出，Greedy 将平均访问节点数从 47 降至 37，减少约 21.28%，说明合理的节点排序能够提高剪枝效率。但其运行时间反而高于 FIFO，表明排序本身也会产生额外开销，节点访问数量减少并不一定带来实际运行速度提升。

Reverse 的优化效果则不稳定：在 H=2 和 H=3 时，访问节点数分别比 FIFO 增加 23.08% 和 34.62%；在 H=4 时仅减少 2.13%。这说明简单地反转访问顺序并不能稳定改善剪枝效果。

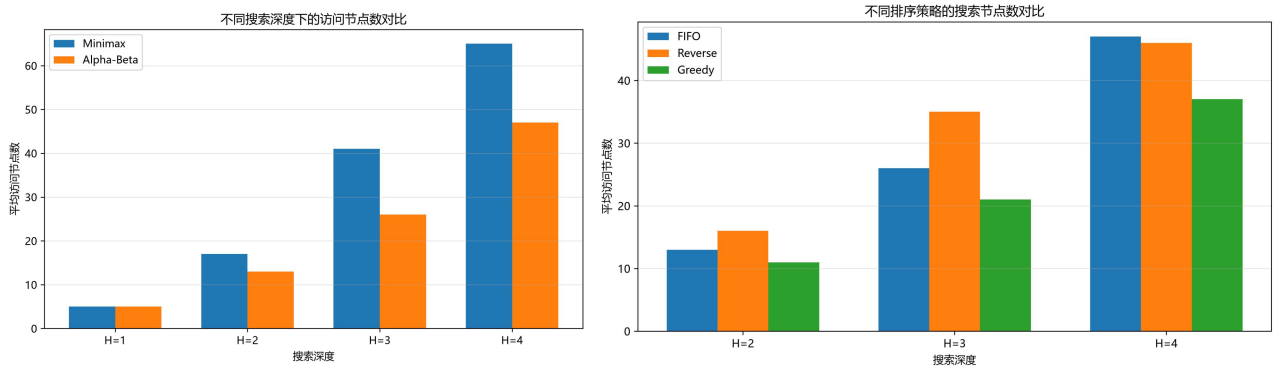


图 5-1 不同搜索深度及节点排序策略的访问节点数对比

图 5-1 进一步展示了不同搜索深度下三种排序策略的整体变化。随着搜索深度增加，Greedy 在各深度下均能减少访问节点，但其运行时间优势并不明显。因此，节点排序优化需要同时考虑剪枝收益与排序开销，不能仅依据访问节点数判断实际性能。

六、实验总结与思考

6.1 优化探索与实验收获

本实验从基础搜索算法实现出发，通过启发函数改进、节点排序和剪枝优化，逐步开展了多组对照实验。实验过程中，我逐渐认识到，算法优化不能只依据理论判断，而需要通过控制实验条件、重复运行和数据对比进行验证。例如，题目三的 Oracle 排序虽然显著减少了搜索节点，但预计算成本使总运行时间增加；题目四的 Greedy 排序减少了 H=4 时的访问节点，却没有带来实际运行速度的提升。这些结果让我认识到，评价算法优化效果不能只关注节点数量，还需要综合考虑计算开销和实际运行时间。

6.2 实验局限与改进方向

本实验部分测试采用固定地图或给定博弈状态，优化结论的适用范围仍有限；性能测试也可能受到运行环境波动的影响。此外，Oracle 排序依赖预先计算真实 Minimax 值，主要用于理想排序效果的参照，并不适合作为实际搜索优化方案。后续可以增加不同随机地图和博弈状态，进一步验证优化策略的稳定性，并改进计时与统计方法，更准确地评估算法收益和额外开销。